BÁO CÁO THỰC HÀNH

Môn học: Blockchain - NT547.Q21.ANTT

Lab 1: Xây dựng Blockchain Cơ bản với Python

GVHD: Trần Tuấn Dũng

1. THÔNG TIN CHUNG:

(Liệt kê tất cả các thành viên trong nhóm)

Lớp: NT547.Q21.ANTT.1

STT	Họ và tên	MSSV	Email
1	Trần Gia Nghi	23521005	23521005@gm.uit.edu.vn

2. NỘI DUNG THỰC HIỆN:[1]

STT	Công việc	Kết quả tự đánh giá
1	Yêu cầu 1	100%
2	Yêu cầu 2	100%
3	Yêu cầu 3	100%
4	Yêu cầu 4	100%
5	Yêu cầu 5 (YC không bắt buộc)	100%
6	Bài tập Bonus 1	100%
7	Bài tập Bonus 2	100%

Phần bên dưới của báo cáo này là tài liệu báo cáo chi tiết của nhóm thực hiện.

[1] Ghi nội dung công việc, các kịch bản trong bài Thực hành

BÁO CÁO CHI TIẾT

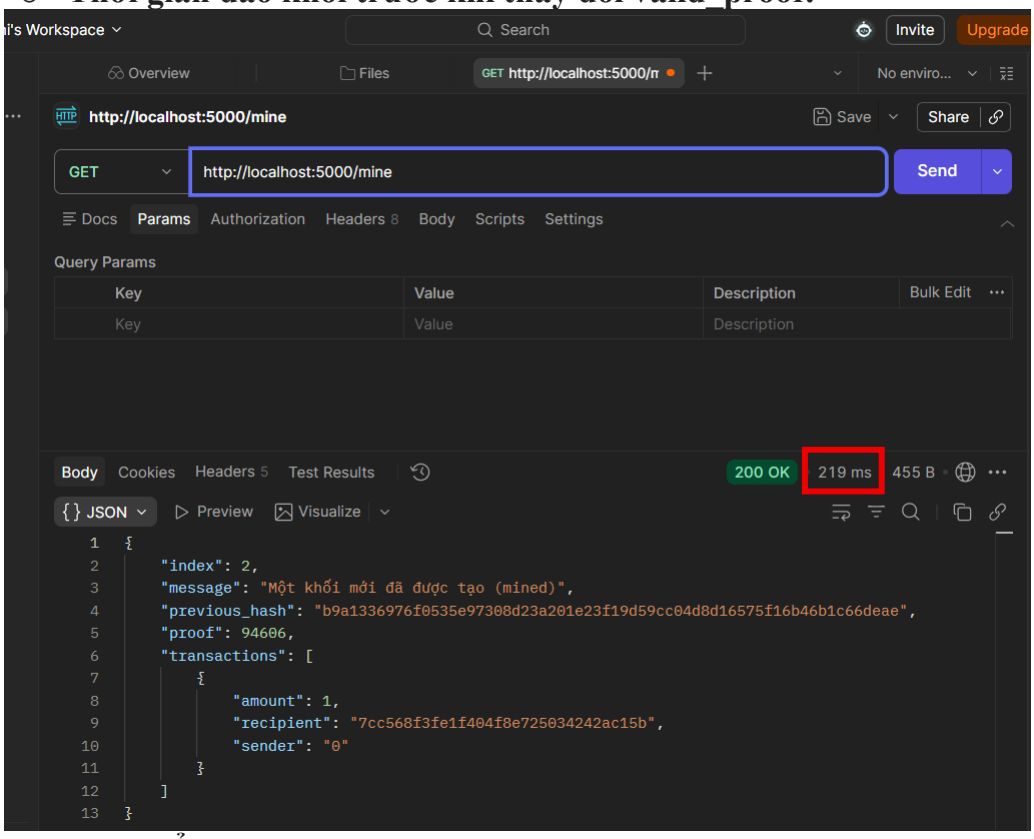
1. Yêu cầu 1: Thay đổi Độ khó của Proof of Work (2.0 điểm)

Mô tả: Trong phương thức `valid_proof`, độ khó hiện tại được quy định bởi việc kiểm tra 4 chữ số 0 ở đầu mã băm (`guess_hash[:4] == "0000"`). Hãy tăng độ khó bằng cách yêu cầu mã băm phải bắt đầu bằng 5 chữ số 0.

Nhiệm vụ:

- Sửa đổi dòng code kiểm tra trong `valid_proof` thành `guess_hash[:5] == "00000"`.
- Chạy lại server và dùng Postman/curl để đào một vài khối.
- Trong báo cáo: Ghi lại và so sánh thời gian đào khối trước và sau khi thay đổi. Giải thích tại sao việc tăng thêm một chữ số 0 lại làm tăng đáng kể thời gian tính toán.

- **Thời gian đào khối trước khi thay đổi valid_proof:**



- **Thay đổi độ khó của PoW:**

```
return guess_hash[:5] == "00000"
```

```python
@staticmethod
def valid_proof(last_proof, proof, last_hash):
    """
    Xác thực bằng chứng: hash(last_proof, proof, last_hash) có chứa 4 số 0 ở đầu không?
    :param last_proof: Proof trước đó
    :param proof: Proof hiện tại
    :param last_hash: Mã băm của khối trước đó
    :return: True nếu đúng, False nếu sai.
    """
    guess = f'{last_proof}{proof}{last_hash}'.encode()
    guess_hash = hashlib.sha256(guess).hexdigest()
    # Thay đổi độ khó bằng cách tăng số lượng số '0' ở đầu
    return guess_hash[:5] == "00000"
```

- **Chạy lại server:**

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL
TERMINAL                                                    python

 * Running on http://172.30.82.205:5000
Press CTRL+C to quit
127.0.0.1 - - [04/Mar/2026 08:27:43] "GET /mine HTTP/1.1" 200 -

D:\notebook_UITNam3\Nam3_ki2\Blockchain\ThucHanh\Blockchain_UIT\Lab1\src>python blockchain.py
 * Serving Flask app 'blockchain'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server inste
ad.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.30.82.205:5000
Press CTRL+C to quit
```

- **Đào lại sau khi thay đổi proof_valid lên guess_hash[:5]**

Overview | Files | GET http://localhost:5000/m | No enviro...

http://localhost:5000/mine Save Share

GET http://localhost:5000/mine Send

Docs Params Authorization Headers 8 Body Scripts Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 5 Test Results 200 OK 3.12 s 457 B

JSON Preview Visualize


```
1  {
2      "index": 3,
3      "message": "Một khối mới đã được tạo (mined)",
4      "previous_hash": "329b626ebae3c47050fbbc16ba50bcaf80c74cfb87393e9a9cadfa54631da583",
5      "proof": 2166075,
6      "transactions": [
7          {
8              "amount": 1,
9              "recipient": "33598e3c632d43359d94460d1e1f5d79",
10             "sender": "0"
11         }
12     ]
13 }
```


Thống kê thời gian đào khối trước và sau khi thay đổi:

Lần	guess_hash[:4]	guess_hash[:5]
1	219 ms	3.12 s
2	225 ms	1.70 s
3	112 ms	463 ms
4	224 ms	1.51 s
5	64 ms	2.99 s

- Việc tăng thêm 1 chữ số 0 làm tăng đáng kể thời gian tính toán (mine)
- Giải thích:
  - Bản chất của hàm hash SHA-256 là trả về một chuỗi ký tự dạng hexadecimal với giá trị mỗi kí tự có 16 trường hợp xảy ra (từ 0 đến 9 và từ a đến f)
  - Thuật toán PoW là thuật toán brute-force cho đến khi tìm ra chuỗi kí tự thỏa mãn điều kiện đầu ra của hàm hash

  ➔ Đối với mỗi kí tự thuật toán PoW sẽ có độ khó là $\frac{1}{16}$

  ➔ Xác suất giải được đối với hàm hash 4 chữ số là $(\frac{1}{16})^4$, với hàm hash 5 chữ số là $(\frac{1}{16})^5$ → tăng thêm 1 chữ số 0 làm tăng độ khó thuật toán lên gấp 16 lần → dẫn đến thời gian giải cũng tăng tương ứng

2. Yêu cầu 2: Phần thưởng Đào khối Động (2.5 điểm)

Mô tả: Phần thưởng cho thợ đào hiện đang được cố định là 1 coin cho mỗi khối. Trong thực tế (như Bitcoin), phần thưởng này sẽ giảm đi theo thời gian (halving). Hãy triển khai một cơ chế phần thưởng động.

Nhiệm vụ:

- Tạo một phương thức mới trong lớp `Blockchain`, ví dụ `get_mining_reward()`.
- Phương thức này sẽ tính toán phần thưởng dựa trên chiều dài hiện tại của chuỗi (`len(self.chain)`). Ví dụ: Phần thưởng là 1 coin cho 10 khối đầu tiên, 0.5 coin cho 10 khối tiếp theo, 0.25 cho 10 khối sau đó, v.v.
- Cập nhật endpoint `/mine` để gọi phương thức `get_mining_reward()` và sử dụng giá trị trả về làm `amount` trong giao dịch thưởng.

Trong báo cáo: Giải thích logic tính toán phần thưởng của em. Chụp ảnh màn hình kết quả API cho thấy phần thưởng thay đổi sau khi vượt qua một mốc nhất định (ví dụ: sau khối thứ 10).

- Ý tưởng:
  - R : là phần thưởng (coins), R mặc định = 1
  - n: là mức thưởng
  - Công thức: R != $\frac{1}{2^n}$ $(\forall x \geq 0)$
  - Mỗi 10 khối thì thay đổi phần thưởng:
    - Lấy length chain / 10, giá trị số nguyên của kết quả phép chia là mức thưởng n.

- Công thức tính R: R $!= \frac{1}{2^n}$ $(\forall x \geq 0)$

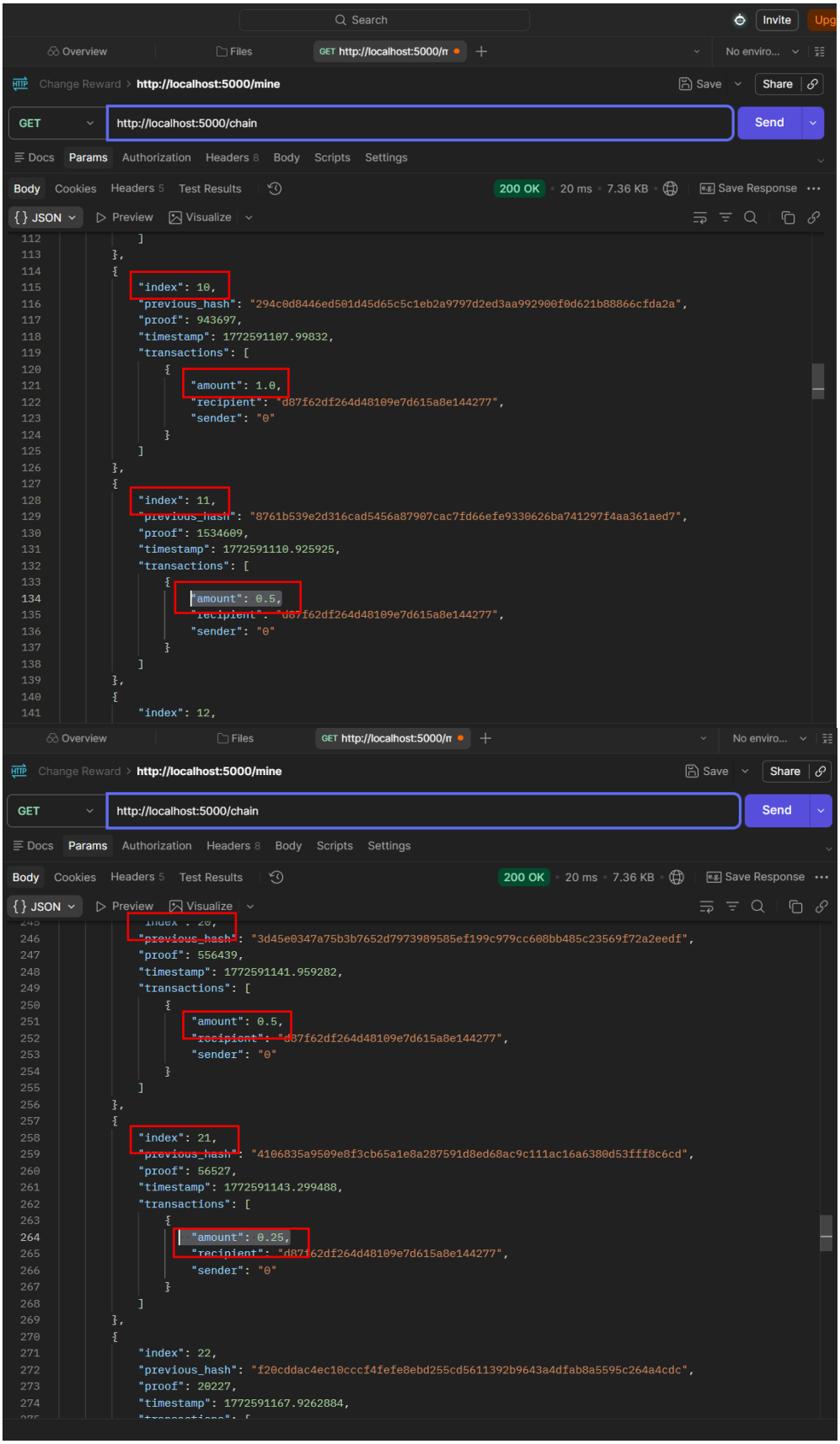
- Code:
  - Hàm get_mining_reward:

```python
class Blockchain:

    def get_mining_reward(self):
        """
        Tính toán phần thưởng đào khối dựa trên chiều dài chuỗi hiện tại.
        Công thức: R = 1 / (2^n) với n là mức thưởng (length // 10)
        """
        # Lấy chiều dài chuỗi chia lấy phần nguyên cho 10 để ra mức thưởng n
        n = len(blockchain.chain) // 10
        # Tính R theo công thức
        R = 1 / (2 ** n)
        return R
```

  - Thay đổi giá trị amount trong API mine để cập nhật giá trị amount theo cơ chế phần thưởng:

```python
@app.route('/mine', methods=['GET'])
def mine():
    # Chạy thuật toán PoW để lấy proof tiếp theo...
    last_block = blockchain.last_block
    proof = blockchain.proof_of_work(last_block)

    # Chúng ta phải nhận phần thưởng cho việc tìm ra proof.
    # Người gửi là "0" để biểu thị rằng node này đã đào một coin mới.
    blockchain.new_transaction(
        sender="0",
        recipient=node_identifier,
        amount=blockchain.get_mining_reward(),
    )

    # Tạo khối mới bằng cách thêm nó vào chuỗi
    previous_hash = blockchain.hash(last_block)
    block = blockchain.new_block(proof, previous_hash)

    response = {
        'message': "Một khối mới đã được tạo (mined)",
        'index': block['index'],
        'transactions': block['transactions'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
    }
    return jsonify(response), 200
```

- Kiểm chứng API với giá trị ‘amount’ động:



3. Yêu cầu 3: Xác thực Giao dịch Cơ bản (3.0 điểm)

Mô tả: Hiện tại, hệ thống cho phép bất kỳ ai gửi một số lượng coin bất kỳ mà không kiểm tra xem họ có đủ số dư hay không. Hãy thêm logic để xác thực giao dịch.

Nhiệm vụ:

- Tạo một phương thức mới `get_balance(address)` để tính toán số dư của một địa chỉ. Phương thức này phải duyệt qua tất cả các khối và các giao dịch trong chuỗi để tính tổng số coin nhận và gửi của địa chỉ đó. Lưu ý: các giao dịch thưởng (sender="0") là coin được tạo ra.
- Trong phương thức `new_transaction`, trước khi thêm giao dịch vào `self.current_transactions`, hãy gọi `get_balance()` để kiểm tra số dư của `sender`.
- Nếu số dư không đủ, giao dịch không được thêm vào và endpoint `/transactions/new` nên trả về một lỗi (ví dụ: HTTP 400 Bad Request với thông báo "Số dư không đủ").

Trong báo cáo: Giải thích cách em triển khai `get_balance`. Chụp ảnh màn hình các trường hợp: một giao dịch thành công, và một giao dịch thất bại do không đủ số dư.

- **Ý tưởng**
  - **Hàm get_balance:** Tính số coin hiện tại mà người gửi có:
    - Duyệt qua tất cả các khối trong chuỗi (for transaction in block['transactions']) và trong các giao dịch (for transaction in self.current_transactions) đang chờ:
      - + amount (coins) khi có địa chỉ là “recipient”
      - - amount (coins) khi có địa chỉ là “sender”
      - Riêng với giao dịch thưởng (sender = “0”) được tính vào số dư của người nhận

```python
################### YEU CAU 3 #######################
    def get_balance(self, address):
        """
        Tính toán số dư của một địa chỉ bằng cách duyệt qua tất cả các khối và giao dịch.
        :param address: Địa chỉ cần kiểm tra số dư
        :return: Số dư của địa chỉ
        """
        balance = 0

        # Duyệt qua tất cả các khối trong chuỗi
        for block in self.chain:
            for transaction in block['transactions']:
                # Nếu địa chỉ là người nhận, cộng số coin
                if transaction['recipient'] == address:
                    balance += transaction['amount']
                # Nếu địa chỉ là người gửi (và không phải giao dịch thưởng), trừ số coin
                if transaction['sender'] == address:
                    balance -= transaction['amount']

        # Duyệt qua các giao dịch đang chờ (chưa được đào)
        for transaction in self.current_transactions:
            if transaction['recipient'] == address:
                balance += transaction['amount']
            if transaction['sender'] == address:
                balance -= transaction['amount']

        return balance
####################### YEU CAU 3 #######################
```

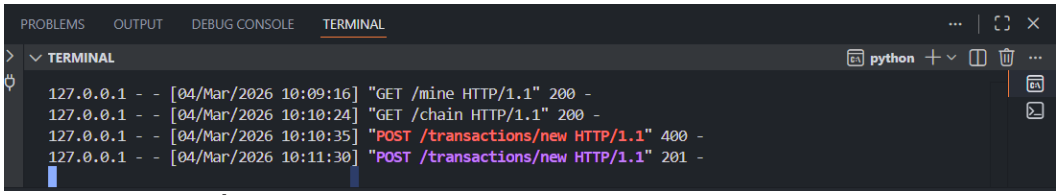
- Trong hàm tạo new_transaction(...):
  - Nếu sender != "0": kiểm tra số dư trước khi thêm giao dịch
  - Nếu số dư < amount: trả về -1 (không thêm giao dịch)

```python
######################## YEU CAU 3 ########################
    def new_transaction(self, sender, recipient, amount):
        """
        Tạo một giao dịch mới để đi vào khối được đào tiếp theo.
        :param sender: Địa chỉ của người gửi
        :param recipient: Địa chỉ của người nhận
        :param amount: Số lượng
        :return: Chỉ số của khối sẽ chứa giao dịch này
        """
######################## YEU CAU 3 ########################
        # Kiểm tra số dư nếu không phải giao dịch thưởng (sender != "0")
        if sender != "0":
            balance = self.get_balance(sender)
            if balance < amount:
                return -1  # Số dư không đủ
######################## YEU CAU 3 ########################
        self.current_transactions.append({
            'sender': sender,
            'recipient': recipient,
            'amount': amount,
        })

        return self.last_block['index'] + 1

```

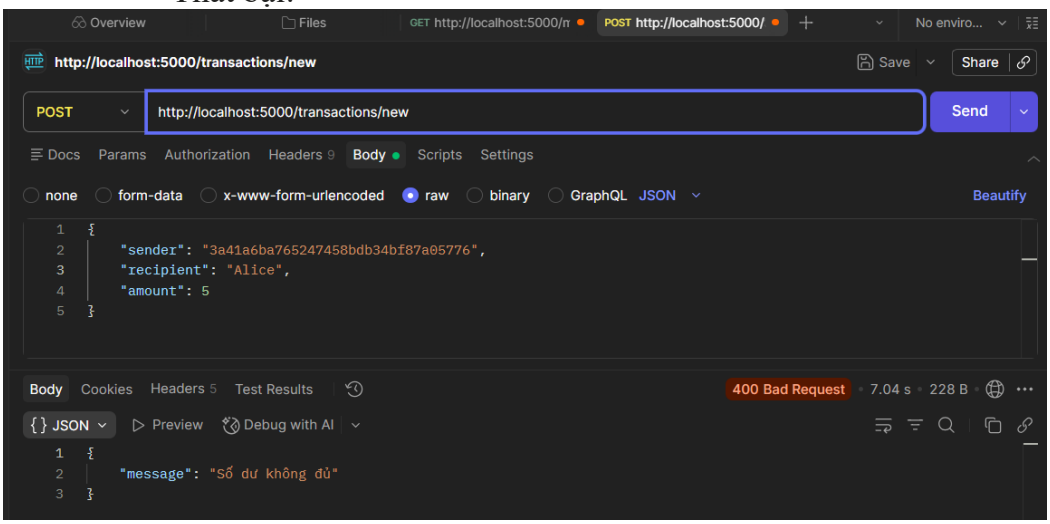
- Endpoint /transactions/new:
  - Kiểm tra nếu index == -1: trả về HTTP 400 với thông báo "Số dư không đủ"

```python
@app.route('/transactions/new', methods=['POST'])
def new_transaction():
    values = request.get_json()

    # Kiểm tra các trường bắt buộc có trong dữ liệu POST không
    required = ['sender', 'recipient', 'amount']
    if not all(k in values for k in required):
        return 'Thiếu giá trị', 400

    # Tạo một giao dịch mới
    index = blockchain.new_transaction(values['sender'], values['recipient'], values['amount'])
######################## YEU CAU 3 ########################
    # Kiểm tra nếu giao dịch thất bại do số dư không đủ
    if index == -1:
        response = {'message': 'Số dư không đủ'}
        return jsonify(response), 400
######################## YEU CAU 3 ########################
    response = {'message': f'Giao dịch sẽ được thêm vào khối {index}'}
    return jsonify(response), 201

```

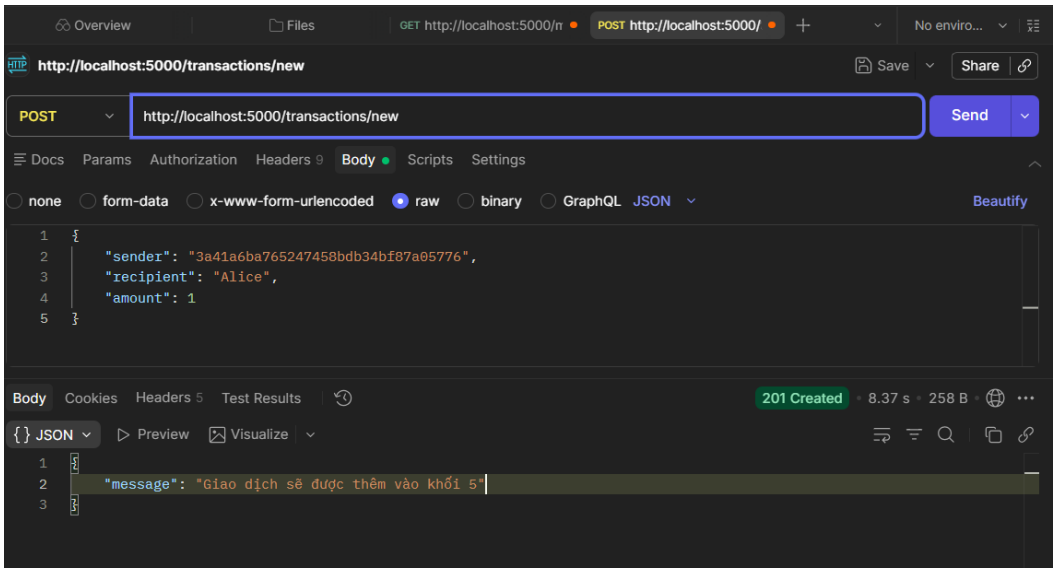
- **Giao dịch:**



- Thất bại:



- Thành công:



4. Yêu cầu 4: Triển khai Cơ chế Đồng thuận (2.5 điểm)

Mô tả: Một blockchain phi tập trung cần một cách để giải quyết xung đột khi các node khác nhau có các phiên bản chuỗi khác nhau. Cơ chế đơn giản nhất là "chuỗi dài nhất sẽ thắng". Mã nguồn cung cấp đã có sẵn các phương thức `register_node`, `valid_chain` và `resolve_conflicts`, nhiệm vụ của em là hiểu và sử dụng chúng.

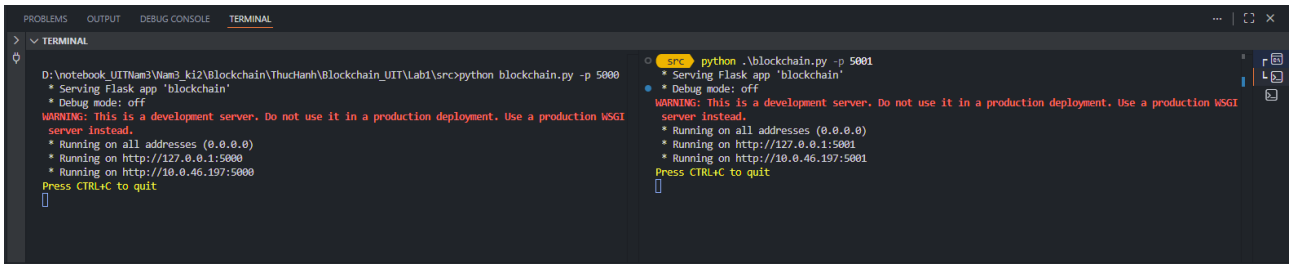
Nhiệm vụ:

- Chạy hai hoặc nhiều node của blockchain trên các cổng khác nhau (ví dụ: `python blockchain.py -p 5000` và `python blockchain.py -p 5001`).
- Dùng endpoint `/nodes/register` để đăng ký node này với node kia và ngược lại.
- Trên node 1, đào một vài khối. Trên node 2, đào nhiều khối hơn.

❖ Gọi endpoint `/nodes/resolve` trên node 1. Quan sát cách chuỗi của node 1 được thay thế bằng chuỗi dài hơn từ node 2. **Trong báo cáo:** Mô tả chi tiết các bước em đã thực hiện để mô phỏng và giải quyết xung đột. Chụp ảnh màn hình API cho thấy chuỗi của node 1 trước và sau khi gọi `/nodes/resolve`. Giải thích tầm quan trọng của cơ chế đồng thuận.

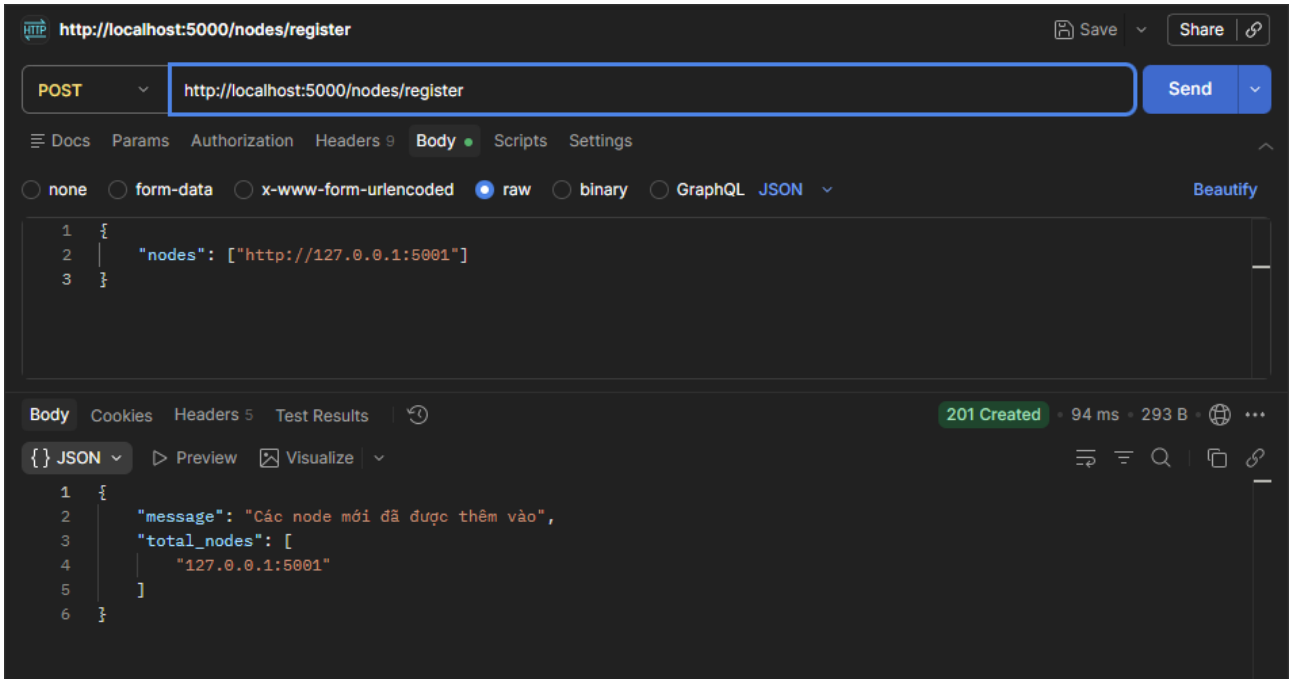
- **Triển khai cơ chế đồng thuận:**
  - **Mở 2 terminal để chạy 2 instance của ứng dụng Flask trên 2 cổng khác nhau: 5000 (Node 1) và 5001 (Node 2).**

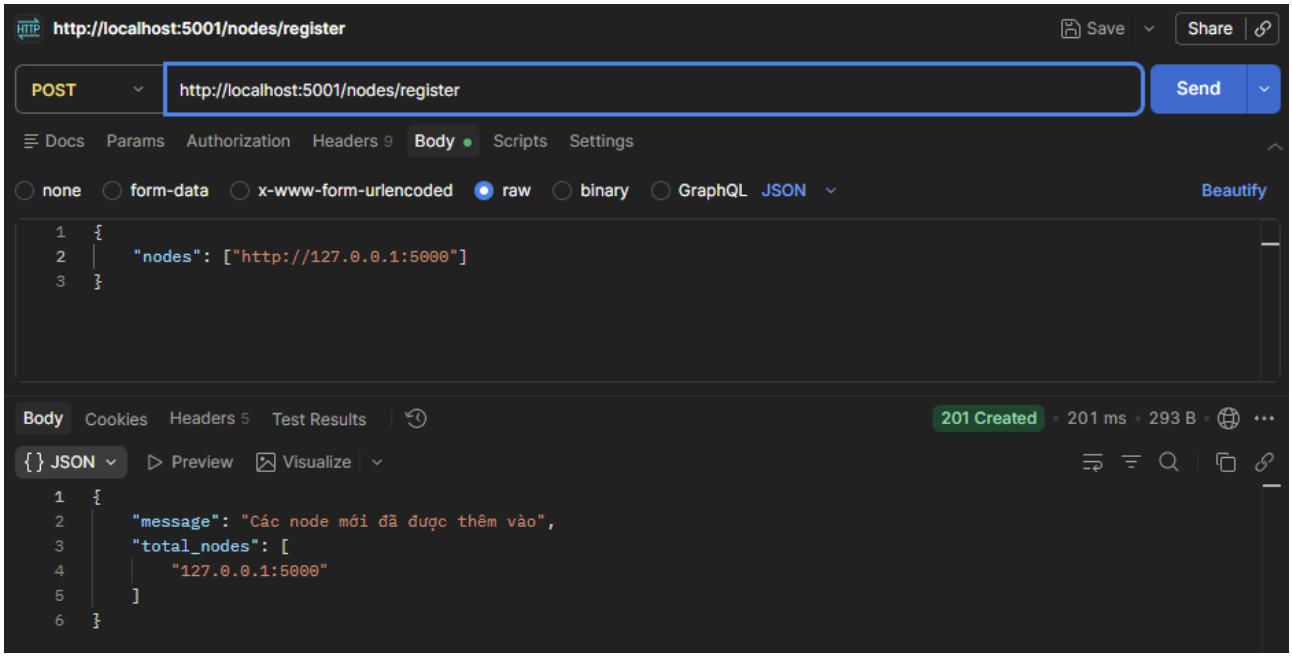
python blockchain.py -p 5000	python blockchain.py -p 5001



  - **Dùng endpoint `/nodes/register` để đăng ký node này với node kia và ngược lại:**

POST: http://localhost:5000/nodes/register
Body:
{
"nodes": ["http://127.0.0.1:5001"]
}
	POST: http://localhost:5001/nodes/register
	Body:
	{
	"nodes": ["http://127.0.0.1:5000"]
	}





- **Trên node 1, đào một vài khối. Trên node 2, đào nhiều khối hơn:** Trên node 1 đào 1 khối, trên node 2 đào 4 khối

❖ **Node 1:** http://localhost:5000/chain → Chain 5000 hiện tại có 2 khối

Body Cookies Headers 5 Test Results 200 OK 71 ms 514 B

{} JSON Preview Visualize


```
{
    "chain": [
        {
            "index": 1,
            "previous_hash": "1",
            "proof": 100,
            "timestamp": 1772631074.6542878,
            "transactions": []
        },
        {
            "index": 2,
            "previous_hash": "fa25e27050cae04ea80ffdfc86efc5de7056b9da62eb19cc77789b24b58b78b6",
            "proof": 467869,
            "timestamp": 1772631329.112869,
            "transactions": [
                {
                    "amount": 1.0,
                    "recipient": "7a418a12366d428f994b3310b2479bd7",
                    "sender": "0"
                }
            ]
        }
    ],
    "length": 2
}
```


- **Node 2:** http://localhost:5001/chain → Chain 5001 hiện tại có 2 khối

Body Cookies Headers 5 Test Results

{} JSON Preview Visualize


```
23        {
24            "index": 3,
25            "previous_hash": "a2903e30c6b873d2798352b13036317f33d17444d813df8110582197eb453af0",
26            "proof": 271420,
27            "timestamp": 1772631348.104167,
28            "transactions": [
29                {
30                    "amount": 1.0,
31                    "recipient": "cca7f33b3d25432d9aaf16f97b494dd7",
32                    "sender": "0"
33                }
34            ]
35        },
36        {
37            "index": 4,
38            "previous_hash": "a7623ed41fe9691ade13a7dabb74a898adb513508f0b80964a28750dc3412186",
39            "proof": 1863328,
40            "timestamp": 1772631360.0362556,
41            "transactions": [
42                {
43                    "amount": 1.0,
44                    "recipient": "cca7f33b3d25432d9aaf16f97b494dd7",
45                    "sender": "0"
46                }
47            ]
48        },
49        {
50            "index": 5,
51            "previous_hash": "77ac1ddc0573dc532847c2ad7180050afc0ff7b470e5d9afefe5e1fa6804473c",
52            "proof": 590689,
53            "timestamp": 1772631433.125365,
54            "transactions": [
55                {
56                    "amount": 1.0,
57                    "recipient": "cca7f33b3d25432d9aaf16f97b494dd7",
58                    "sender": "0"
59                }
60            ]
61        }
62    ],
63    "length": 5
64 }
```


- **Gọi endpoint `/nodes/resolve` trên node 1. Quan sát cách chuỗi của node 1 được thay thế bằng chuỗi dài hơn từ node 2:**

http://localhost:5001/nodes/resolve

Overview | Files | GET http://localhost:5000/n | GET http://localhost:5001/n

http://localhost:5001/nodes/resolve

GET http://localhost:5001/nodes/resolve

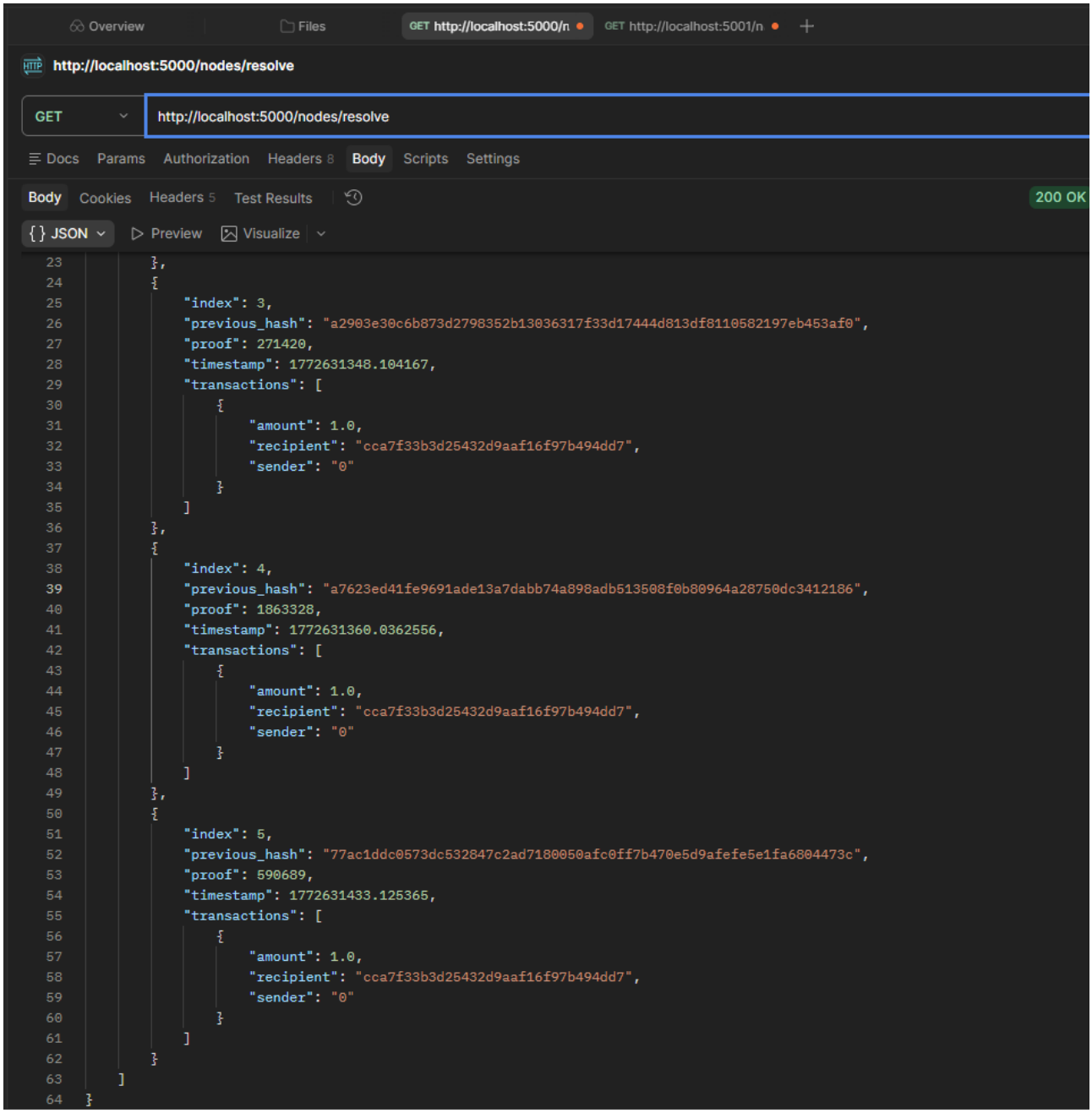
Docs | Params | Authorization | Headers 9 | Body | Scripts | Settings

Body | Cookies | Headers 5 | Test Results

{ } JSON | Preview | Visualize


```
        {
            "index": 3,
            "previous_hash": "a2903e30c6b873d2798352b13036317f33d17444d813df8110582197eb453af0",
            "proof": 271420,
            "timestamp": 1772631348.104167,
            "transactions": [
                {
                    "amount": 1.0,
                    "recipient": "cca7f33b3d25432d9aaf16f97b494dd7",
                    "sender": "0"
                }
            ]
        },
        {
            "index": 4,
            "previous_hash": "a7623ed41fe9691ade13a7dabb74a898adb513508f0b80964a28750dc3412186",
            "proof": 1863328,
            "timestamp": 1772631360.0362556,
            "transactions": [
                {
                    "amount": 1.0,
                    "recipient": "cca7f33b3d25432d9aaf16f97b494dd7",
                    "sender": "0"
                }
            ]
        },
        {
            "index": 5,
            "previous_hash": "77ac1ddc0573dc532847c2ad7180050afc0ff7b470e5d9afefe5e1fa6804473c",
            "proof": 590689,
            "timestamp": 1772631433.125365,
            "transactions": [
                {
                    "amount": 1.0,
                    "recipient": "cca7f33b3d25432d9aaf16f97b494dd7",
                    "sender": "0"
                }
            ]
        }
    ],
    "message": "Chuỗi của chúng ta là chính xác"
}
```


http://localhost:5000/nodes/resolve




```
23          },
24          {
25              "index": 3,
26              "previous_hash": "a2903e30c6b873d2798352b13036317f33d17444d813df8110582197eb453af0",
27              "proof": 271420,
28              "timestamp": 1772631348.104167,
29              "transactions": [
30                  {
31                      "amount": 1.0,
32                      "recipient": "cca7f33b3d25432d9aaf16f97b494dd7",
33                      "sender": "0"
34                  }
35              ]
36          },
37          {
38              "index": 4,
39              "previous_hash": "a7623ed41fe9691ade13a7dabb74a898adb513508f0b80964a28750dc3412186",
40              "proof": 1863328,
41              "timestamp": 1772631360.0362556,
42              "transactions": [
43                  {
44                      "amount": 1.0,
45                      "recipient": "cca7f33b3d25432d9aaf16f97b494dd7",
46                      "sender": "0"
47                  }
48              ]
49          },
50          {
51              "index": 5,
52              "previous_hash": "77ac1ddc0573dc532847c2ad7180050afc0ff7b470e5d9afefe5e1fa6804473c",
53              "proof": 590689,
54              "timestamp": 1772631433.125365,
55              "transactions": [
56                  {
57                      "amount": 1.0,
58                      "recipient": "cca7f33b3d25432d9aaf16f97b494dd7",
59                      "sender": "0"
60                  }
61              ]
62          }
63      ]
64  }
```


➔ Khi GET http://localhost:5000/nodes/resolve, node 1 (localhost:5000) đi hỏi các node xung quanh (node 2 - localhost:5001). Khi thấy chain của node 2 dài hơn chain node 1 → Node 1 sẽ vứt bỏ chain hiện tại và copy toàn bộ chain của node 2

Giải thích tầm quan trọng của cơ chế đồng thuận trên Blockchain:

Blockchain ra đời để cải thiện điểm yếu single points of failure trên các hệ thống truyền thống. Để làm được việc đó, Blockchain là một mạng lưới phân tán đảm bảo các tính immutability, privacy, security, và transparency. Cớ chế đồng thuận là thành phần cốt lõi của blockchain để duy trì các tính chất đó, với tầm quan trọng:

- Duy trì một nguồn sự thật duy nhất (Maintains a Single Source of Truth): Trong một mạng lưới phân tán, các nút (node) khác nhau có thể nhận dữ liệu

giao dịch ở các thời điểm khác nhau. Cơ chế đồng thuận đảm bảo tất cả các nút đều đi đến một thỏa thuận chung để xác định đâu là khối dữ liệu hợp lệ duy nhất được thêm vào chuỗi, tạo ra một phiên bản sổ cái nhất quán.

- Ngăn chặn chi tiêu kép (Prevents Double-Spending): Đối với các loại tiền điện tử như Bitcoin, cơ chế đồng thuận đóng vai trò nền tảng trong việc xác minh thứ tự giao dịch, đảm bảo rằng một người dùng không thể tiêu cùng một đồng tiền hai lần.
- Đảm bảo bảo mật mà không cần sự tin cậy (Ensures Security Without Trust): Blockchain hoạt động dựa trên nguyên tắc "trustless", các bên tham gia không cần phải biết hay tin tưởng lẫn nhau. Cơ chế đồng thuận thiết lập niềm tin giữa các đồng đẳng vô danh trong môi trường máy tính phân tán, đảm bảo rằng các tác nhân độc hại không thể dễ dàng thao túng hệ thống.
- Hiện thực hóa tính phi tập trung (Enables Decentralization): Thay vì tập trung quyền lực vào một thực thể duy nhất, cơ chế đồng thuận phân phối quyền ra quyết định (như ai được xác thực giao dịch, ai được thêm khối mới) trên hàng ngàn nút mạng. Điều này làm cho blockchain trở nên phi tập trung và có khả năng chống kiểm duyệt.
- Đảm bảo khả năng chịu lỗi (Fault Tolerance): Nó đảm bảo rằng toàn bộ hệ thống vẫn có thể tiếp tục hoạt động trơn tru ngay cả khi một số phần tử trong mạng lưới gặp sự cố kết nối hoặc cố tình có những hành vi gian lận (ví dụ như thuật toán Practical Byzantine Fault Tolerance)

Ref: https://www.geeksforgeeks.org/compiler-design/consensus-algorithms-in-blockchain/, https://crypture.org/why-do-blockchains-require-consensus-mechanisms-understanding-the-backbone-of-trust-in-decentralized-networks/

5. Yêu cầu 5: Ký và Xác thực Giao dịch bằng Chữ ký số (Nâng cao - Extra 2.0 điểm)

Mô tả: Đây là yêu cầu không bắt buộc để đạt điểm tối đa (10), nhưng sẽ giúp em có thêm điểm thưởng. Để đảm bảo tính xác thực, mỗi giao dịch cần được ký bởi khóa riêng (private key) của người gửi. Các node khác sau đó có thể xác minh giao dịch bằng khóa công khai (public key) của người gửi.

Nhiệm vụ:

- Sử dụng một thư viện mật mã như `pycryptodome` (`pip install pycryptodome`).
- Tạo một endpoint mới, ví dụ `/wallet/new`, để tạo một cặp khóa public/private mới.
- Sửa đổi endpoint `/transactions/new` để yêu cầu người dùng cung cấp `sender_public_key` và `signature` cùng với các thông tin giao dịch.
- Viết một phương thức `verify_signature` để kiểm tra xem chữ ký có hợp lệ với giao dịch và khóa công khai của người gửi hay không.
- Chỉ thêm giao dịch vào danh sách nếu chữ ký hợp lệ.

Trong báo cáo: Trình bày cách em tích hợp hệ thống chữ ký số. Giải thích vai trò của khóa công khai và khóa riêng. Cung cấp ví dụ về một yêu cầu API hoàn chỉnh (bao gồm cả chữ ký) để tạo giao dịch.

❖ **Triển khai:**

Hệ thống được chia thành 3 phần xử lý chính (thông qua 3 API Endpoints):

- **Khởi tạo Ví (Tạo cặp khóa RSA)**
  - **API:** GET /wallet/new
  - **Logic:** Tạo ngẫu nhiên Private Key (1024-bit) → Suy ra Public Key → Chuyển mã cả hai sang chuỗi Hex → Trả JSON về cho người dùng:
    - Bước 1: Sinh cặp khóa RSA: Lệnh RSA.generate(1024) sẽ tạo ngẫu nhiên một Khóa riêng tư (Private Key) bảo mật có độ dài 1024 bit. Từ khóa riêng tư gốc này, lệnh private_key.publickey() sẽ tự động tính toán và trích xuất ra Khóa công khai (Public Key) tương ứng. Hai khóa này giờ đây tạo thành một cặp không thể tách rời.
    - Bước 2: Định dạng (Format) dữ liệu để truyền tải: Các khóa RSA vừa tạo ban đầu tồn tại dưới dạng đối tượng toán học của Python. Code sử dụng export_key(format='DER') để xuất chúng thành dữ liệu nhị phân (byte thô). Sau đó, lệnh binascii.hexlify được dùng để dịch các byte này sang dạng chuỗi Hex (hệ cơ số 16).
    - Bước 3: Trả về kết quả: Đóng gói cả hai chuỗi private_key và public_key vừa được định dạng vào một cấu trúc JSON, sau đó trả về cho phần mềm/người dùng đang gọi API kèm theo mã trạng thái HTTP 200 (Thành công).

```
@app.route('/wallet/new', methods=['GET'])
def new_wallet():
    """Tạo một cặp khóa Public/Private Key RSA mới"""
    private_key = RSA.generate(1024)
    public_key = private_key.publickey()

    response = {
        'private_key': binascii.hexlify(private_key.export_key(format='DER')).decode('ascii'),
        'public_key': binascii.hexlify(public_key.export_key(format='DER')).decode('ascii')
    }
    return jsonify(response), 200
```

- **Ký Giao dịch (Tạo Chữ ký số)**
  - **API:** POST /wallet/sign
  - **Logic:** Lấy dữ liệu giao dịch → Băm bằng SHA-256 → Dùng Private Key để ký bản băm đó → Trả về chữ ký:

- Bước 1: Kiểm tra đầu vào: Nhận dữ liệu request và xác minh xem đã có đủ 4 thông tin bắt buộc chưa (private_key, sender, recipient, amount). Nếu thiếu bất kỳ trường nào, lập tức báo lỗi 400.
- Bước 2: Tạo bản băm (Hashing): Gom các thông tin giao dịch (người gửi, người nhận, số tiền) ghép lại thành một chuỗi văn bản liền mạch. Sau đó, dùng thuật toán SHA-256 để băm chuỗi này thành một bản tóm tắt dữ liệu (h).
- Bước 3: Ký giao dịch: Chuyển đổi khóa riêng tư (private_key) từ dạng chuỗi Hex sang định dạng khóa RSA chuẩn. Sau đó, dùng chính khóa này để mã hóa (ký) bản băm h bằng thuật toán pkcs1_15. Kết quả thu được là một chữ ký số độc nhất.
- Bước 4: Trả kết quả: Đổi chữ ký vừa tạo sang chuỗi Hex để dễ đọc/truyền tải qua mạng và trả về cho người dùng (mã 200). Nếu khóa private key đưa vào bị sai định dạng hoặc không hợp lệ, hệ thống sẽ bắt lỗi (except) và thông báo thất bại (mã 400).

```
@app.route('/wallet/sign', methods=['POST'])
def sign_transaction():
    values = request.get_json()
    required = ['private_key', 'sender', 'recipient', 'amount']
    if not all(k in values for k in required):
        return 'Thiếu giá trị', 400

    private_key_hex = values['private_key']
    sender = values['sender']
    recipient = values['recipient']
    amount = values['amount']

    transaction_string = f"{sender}{recipient}{amount}"
    h = SHA256.new(transaction_string.encode('utf8'))

    try:
        private_key = RSA.import_key(binascii.unhexlify(private_key_hex))
        signature = pkcs1_15.new(private_key).sign(h)
        signature_hex = binascii.hexlify(signature).decode('ascii')

        return jsonify({'signature': signature_hex}), 200
    except Exception as e:
        return jsonify({'message': 'Lỗi khi ký', 'error': str(e)}), 400
```

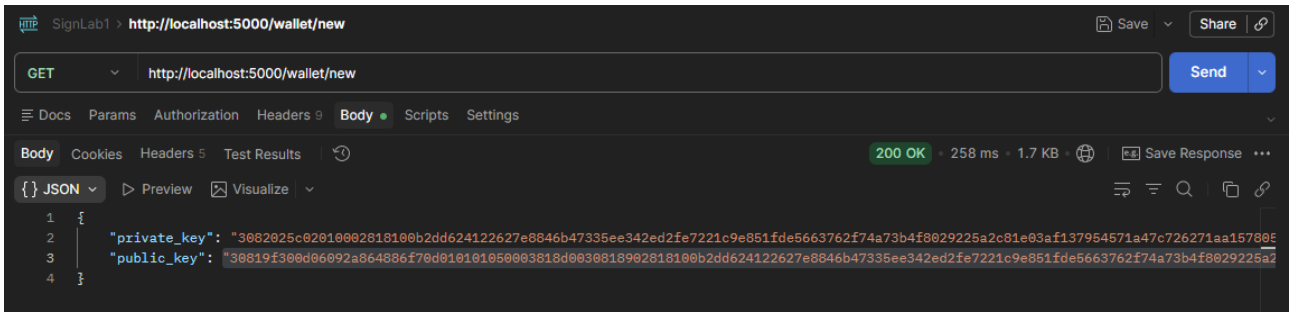
- **Xác thực và Đưa vào Blockchain**
  - **API:** POST /transactions/new
  - **Logic:** Đóng vai trò là Node mạng lưới. Trước khi đưa giao dịch vào danh sách chờ (mempool), Node sẽ yêu cầu người gửi cung cấp cả thông tin giao dịch, signature, và sender_public_key.

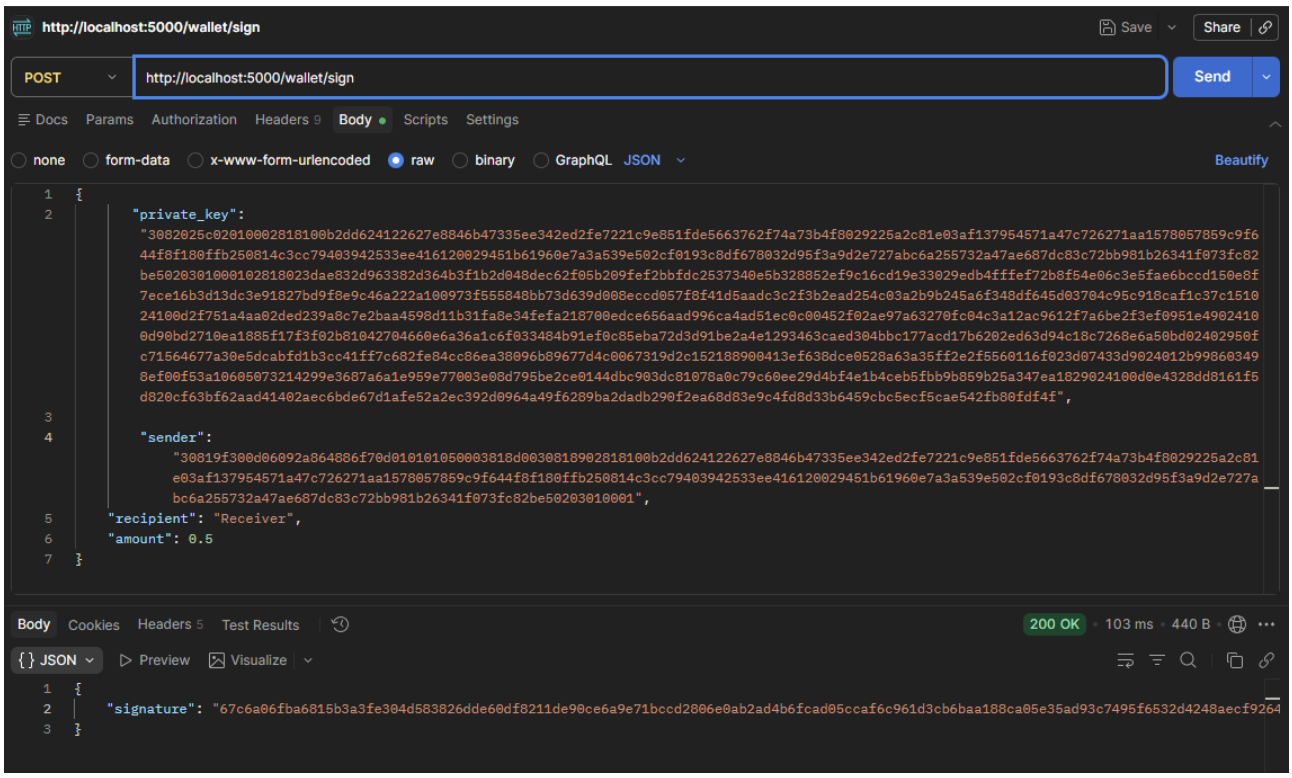
```python
@app.route('/transactions/new', methods=['POST'])
def new_transaction():
    values = request.get_json()

    required = ['sender', 'recipient', 'amount', 'signature', 'sender_public_key']
    if not all(k in values for k in required):
        return 'Thiếu giá trị', 400

    sender = values['sender']
    recipient = values['recipient']
    amount = values['amount']
    signature = values['signature']
    sender_pub_key = values['sender_public_key']

    # Chuỗi dữ liệu phải khớp 100% với lúc ký ở API /wallet/sign
    transaction_string = f"{sender}{recipient}{amount}"

    # Xác thực chữ ký
    if not blockchain.verify_signature(sender_pub_key, signature, transaction_string):
        return jsonify({'message': 'Lỗi: Chữ ký không hợp lệ!'}), 400

    # Nếu hợp lệ, đưa giao dịch vào blockchain
    index = blockchain.new_transaction(sender, recipient, amount)

    if index is False:
        return jsonify({'message': 'Số dư không đủ'}), 400

    response = {'message': f'Giao dịch hợp lệ và sẽ được thêm vào khối {index}'}
    return jsonify(response), 201
```

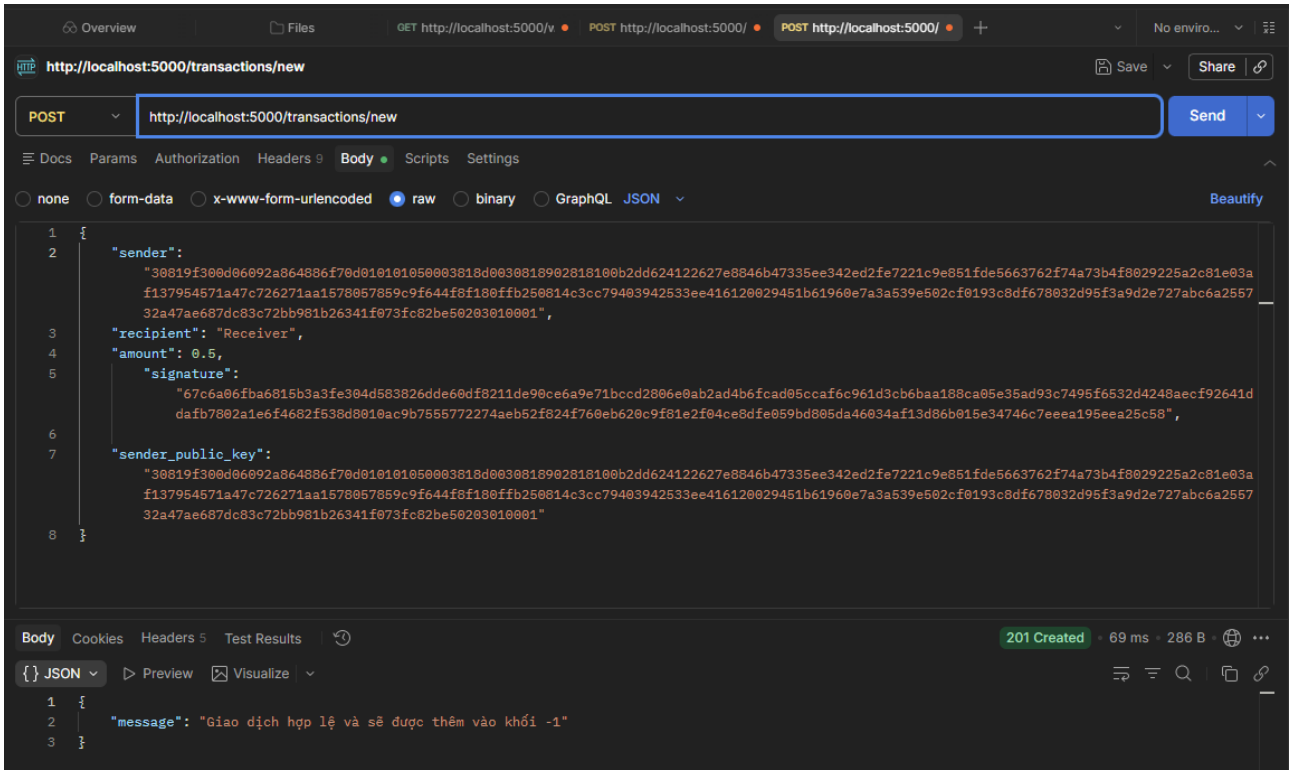
- Node gọi phương thức verify_signature:
  - Khôi phục Public Key: Chuyển đổi khóa công khai của người gửi (sender_public_key) từ chuỗi Hex sang đối tượng RSA chuẩn để thư viện toán học có thể sử dụng.
  - Tạo bản băm (Hashing): Băm nội dung của giao dịch (transaction_string) bằng thuật toán SHA-256. Quá trình xác thực sẽ kiểm tra trên bản băm này thay vì toàn bộ chuỗi văn bản gốc.
  - Đối chiếu chữ ký: Sử dụng thuật toán pkcs1_15, lấy Public Key để "giải mã" chữ ký (signature). Nếu nội dung bên trong chữ ký khớp hoàn toàn với bản băm SHA-256 vừa tạo ở bước 2, nghĩa là giao dịch do chính chủ ký và chưa bị chỉnh sửa thì trả về True.
  - Từ chối nếu sai lệch: Nếu có bất kỳ điểm bất thường nào (chữ ký giả mạo, nội dung giao dịch bị ai đó sửa đổi làm bản băm thay đổi), hàm verify sẽ sinh ra lỗi (ValueError hoặc TypeError). Khối except sẽ bắt lấy lỗi này và trả về False (từ chối giao dịch).

```python
@staticmethod
def verify_signature(sender_public_key, signature, transaction_string):
    """
    Kiểm tra chữ ký số của giao dịch.
    """
    try:
        # Chuyển đổi public key từ dạng hex sang đối tượng RSA
        public_key = RSA.import_key(binascii.unhexlify(sender_public_key))
        # Băm chuỗi dữ liệu giao dịch
        h = SHA256.new(transaction_string.encode('utf8'))
        # Xác thực chữ ký
        pkcs1_15.new(public_key).verify(h, binascii.unhexlify(signature))
        return True
    except (ValueError, TypeError):
        return False
```

❖ **Chạy thử:**







❖ **Giải thích vai trò của khóa công khai và khóa riêng:**

- **Private Key (Khóa riêng tư)**
  - Là một mảng byte sinh ra từ các số nguyên tố cực lớn (trong code của bạn là RSA 1024-bit). Nó được quản lý và lưu trữ hoàn toàn ở Client-side (ứng dụng Ví của người dùng), tuyệt đối không bao giờ được gửi qua API network.
  - Vai trò xử lý: Dùng để thực thi tác vụ Ký số (Signing). Khi user muốn đổi state của Blockchain (chuyển coin), Private Key sẽ mã hóa bản băm SHA-256 của raw payload (sender + recipient + amount) thành một chuỗi signature.
  - Bài toán giải quyết:
    - Authentication (Xác thực): Chứng minh client gọi API thực sự là owner của cái ví đó.
    - Non-repudiation (Chống chối bỏ): Vì thuật toán RSA đảm bảo chỉ có hàm chứa Private Key mới sinh ra được signature đó, client không thể gửi request lên rồi sau đó cãi là "tôi không gửi".
- **Public Key (Khóa công khai)**
  - Bản chất kỹ thuật: Được dẫn xuất (derive) trực tiếp từ Private Key theo thuật toán toán học một chiều. Trong code của bạn, nó được mã hóa Hex và dùng luôn làm định danh hệ thống (trường sender / Wallet Address). Nó là Public Data, ai cũng có thể xem.
  - Vai trò xử lý: Dùng ở Server-side (các Node) để thực thi tác vụ Xác minh (Verification). Node sẽ chạy thuật toán pkcs1_15.verify(): dùng Public Key để giải mã signature lấy ra mã băm gốc, sau đó đối chiếu với mã băm SHA-256 do Node tự tính từ payload nhận được.

- Bài toán giải quyết:
  - Data Integrity (Tính toàn vẹn dữ liệu): Ngăn chặn triệt để tấn công Man-in-the-Middle (MitM). Nếu hacker chặn request ở giữa và sửa amount từ 0.5 thành 100, mã băm tại Node sẽ thay đổi, hàm verify lập tức quăng lỗi ValueError vì hash không khớp với signature.
  - Stateless Verification (Xác thực phi trạng thái): Đảm bảo tính phi tập trung (Decentralized). Các Node không cần query vào một Database tập trung nào để check session hay kiểm tra xem user này là ai; chỉ cần dùng toán học thuần túy là xác minh được tính hợp lệ của request HTTP.

6. Bài tập Bonus 1:

Mô tả:

- Sửa endpoint transactions/new, thêm 1 trường optional là "message". Chèn thêm thông điệp vào Block (giống Nakamoto chèn vào genesis block của Blockchain) (Max: 50 chars)
- Thực hiện 3 transactions có "message"
- Đào khối và gọi /chain để thấy

- **Triển khai:**
  - Sửa hàm tọa new transaction:
    - Thêm biến message là optional:

```
def new_transaction(self, sender, recipient, amount, message=None):
```

- Kiểm tra nếu paremeter message tồn tại thì thêm vào TX:

```
if message:
        transaction['message'] = message[:50]  # Giới hạn 50 ký tự
```

```
def new_transaction(self, sender, recipient, amount, message=None):
    """
    Tạo một giao dịch mới để đi vào khối được đào tiếp theo.
    :param sender: Địa chỉ của người gửi
    :param recipient: Địa chỉ của người nhận
    :param amount: Số lượng
    :return: Chỉ số của khối sẽ chứa giao dịch này
    """
######################### YC3 #########################
    # Kiểm tra số dư nếu không phải giao dịch thưởng (sender != "0")
    if sender != "0":
        balance = self.get_balance(sender)
        if balance < amount:
            return -1  # Số dư không đủ
######################### YC3 #########################
```

```
        transaction = {
            'sender': sender,
            'recipient': recipient,
            'amount': amount,
        }

######################### BONUS 1 #########################
        if message:
            transaction['message'] = message[:50]  # Giới hạn 50 ký tự

        self.current_transactions.append(transaction)

        return self.last_block['index'] + 1
```

- Sửa endpoint @app.route('/transactions/new', methods=['POST']): thêm đoạn lấy message thêm vào tx:

```
    # Lấy thông điệp tùy chọn (max 50 ký tự)
    message = values.get('message')
    if message and len(message) > 50:
        return jsonify({'message': 'Thông điệp không được vượt quá 50 ký tự'}), 400

    # Tạo một giao dịch mới
    index = blockchain.new_transaction(
        values['sender'],
        values['recipient'],
        values['amount'],
        message=message
    )
```

❖

❖ **Quan sát:**

- **Ban đầu:**

GET http://localhost:8000/chain Send

Docs Params Authorization Headers 9 Body Scripts Settings

Body Cookies Headers 5 Test Results 200 OK 57 ms 749 B Save Response

{ } JSON Preview Visualize


```
{
    "chain": [
        {
            "index": 1,
            "previous_hash": "1",
            "proof": 100,
            "timestamp": 1772640230.4326162,
            "transactions": []
        },
        {
            "index": 2,
            "previous_hash": "5fe545e88e31264d4d2065c0e979f8d85787351ef864deffddf5d66f5ec0f174",
            "proof": 2103419,
            "timestamp": 1772640252.156881,
            "transactions": [
                {
                    "amount": 1.0,
                    "recipient": "010f7ef5675e4b8bb961f85a58aa84d0",
                    "sender": "0"
                }
            ]
        },
        {
            "index": 3,
            "previous_hash": "d3169a19a9ba28fd9bd7476367f3865193ef3728d24e7ccd1e181eac94859684",
            "proof": 823206,
            "timestamp": 1772640263.0612638,
            "transactions": [
                {
                    "amount": 1.0,
                    "recipient": "010f7ef5675e4b8bb961f85a58aa84d0",
                    "sender": "0"
                }
            ]
        }
    ],
    "length": 3
}
```


- **Thực hiện 3 TX có "message"**

http://localhost:8000/transactions/new Save Share

POST http://localhost:8000/transactions/new Send

Docs Params Authorization Headers 9 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify


```
{
    "sender": "010f7ef5675e4b8bb961f85a58aa84d0",
    "recipient": "Rec",
    "amount": 0.1,
    "message": "Test Mess lan 1"
}
```


Body Cookies Headers 5 Test Results 201 Created 45 ms 259 B

{ } JSON Preview Visualize


```
{
    "message": "Giao dịch sẽ được thêm vào khối 4"
}
```

POST http://localhost:8000/transactions/new Send

Docs Params Authorization Headers 9 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
{
    "sender": "010f7ef5675e4b8bb961f85a58aa84d0",
    "recipient": "Rec",
    "amount": 0.1,
    "message": "Test Mess lan 2"
}
```

Body Cookies Headers 5 Test Results 201 Created 126 ms 259 B

{} JSON Preview Visualize

```
{
    "message": "Giao dịch sẽ được thêm vào khối 4"
}
```

❖ /mine để TX chuyển vào chain, Kiểm tra /chain:

GET http://localhost:8000/chain Send

Docs Params Authorization Headers 9 Body Scripts Settings

Body Cookies Headers 5 Test Results 200 OK 68 ms 1.26 KB Save Response

{} JSON Preview Visualize


```
            "transactions": [
                {
                    "amount": 1.0,
                    "recipient": "010f7ef5675e4b8bb961f85a58aa84d0",
                    "sender": "0"
                }
            ]
        },
        {
            "index": 4,
            "previous_hash": "6aa8f5569dea17737008aa2b61d73c07d8454f48f3a9deea822f2cdbc4736452",
            "proof": 41760,
            "timestamp": 1772640400.9576993,
            "transactions": [
                {
                    "amount": 0.1,
                    "message": "Test Mess lan 1",
                    "recipient": "Rec",
                    "sender": "010f7ef5675e4b8bb961f85a58aa84d0"
                },
                {
                    "amount": 0.1,
                    "message": "Test Mess lan 2",
                    "recipient": "Rec",
                    "sender": "010f7ef5675e4b8bb961f85a58aa84d0"
                },
                {
                    "amount": 1,
                    "message": "Test Mess lan 3",
                    "recipient": "Rec",
                    "sender": "010f7ef5675e4b8bb961f85a58aa84d0"
                },
                {
                    "amount": 1.0,
                    "recipient": "010f7ef5675e4b8bb961f85a58aa84d0",
                    "sender": "0"
                }
            ]
        }
    ],
    "length": 4
}
```


7. Bài tập Bonus 2:

Mô tả:

❖ Thêm trường "fee" vào cấu trúc của transactions, khi fee:
Fee cao: ưu tiên xử lý, đưa vào khối trước
Fee thấp: Chờ lâu hơn

❖ **Triển khai:**

- **Hàm new_transaction():**
  - Thêm parameter "fee" mặc định = 0
  - Thêm fee vào transaction dict
  - Sắp xếp current_transactions theo fee giảm dần (cao đến thấp)

```
self.current_transactions.sort(key=lambda x: x.get('fee', 0), reverse=True)
```

```
def new_transaction(self, sender, recipient, amount, message=None, fee=0):
    """
    Tạo một giao dịch mới để đi vào khối được đào tiếp theo.
    :param sender: Địa chỉ của người gửi
    :param recipient: Địa chỉ của người nhận
    :param amount: Số lượng
    :return: Chỉ số của khối sẽ chứa giao dịch này
    """
########################## YC3 ##########################
    # Kiểm tra số dư nếu không phải giao dịch thưởng (sender != "0")
    if sender != "0":
        balance = self.get_balance(sender)
        if balance < amount:
            return -1  # Số dư không đủ
######################### YC3 ##########################

    transaction = {
        'sender': sender,
        'recipient': recipient,
        'amount': amount,
        'fee': fee,  # Phí giao dịch
    }

######################### BONUS 1 ########################
    if message:
        transaction['message'] = message[:50]  # Giới hạn 50 ký tự

    self.current_transactions.append(transaction)

########################## BONUS 2: Sắp xếp theo fee (cao -> thấp) ##########################
    # Fee cao được ưu tiên xử lý trước, fee thấp chờ lâu hơn
    self.current_transactions.sort(key=lambda x: x.get('fee', 0), reverse=True)

    return self.last_block['index'] + 1
```

- **Endpoint /transactions/new:**
  - Nhận thêm trường fee (mặc định = 0)

- Validate: fee không được âm

```
@app.route('/transactions/new', methods=['POST'])
def new_transaction():
    values = request.get_json()

    # Kiểm tra các trường bắt buộc có trong dữ liệu POST không
    required = ['sender', 'recipient', 'amount']
    if not all(k in values for k in required):
        return jsonify({'message': 'Thiếu giá trị: sender, recipient, amount'}), 400

    # Lấy thông điệp tùy chọn (max 50 ký tự)
    message = values.get('message')
    if message and len(message) > 50:
        return jsonify({'message': 'Thông điệp không được vượt quá 50 ký tự'}), 400

    # Lấy phí giao dịch (mặc định = 0)
    fee = values.get('fee', 0)
    if fee < 0:
        return jsonify({'message': 'Phí giao dịch không được âm'}), 400

    # Tạo một giao dịch mới
    index = blockchain.new_transaction(
        values['sender'],
        values['recipient'],
        values['amount'],
        message=message,
        fee=fee
    )

######################### YC3 #########################
    # Kiểm tra nếu giao dịch thất bại do số dư không đủ
    if index == -1:
        response = {'message': 'Số dư không đủ'}
        return jsonify(response), 400
######################### YC3 #########################



    response = {'message': f'Giao dịch sẽ được thêm vào khối {index}'}
    return jsonify(response), 201
❖
```

❖ **Thử nghiệm:**

- **Chain ban đầu:**

http://localhost:5555/chain

GET http://localhost:5555/chain Send

Docs Params Authorization Headers 8 Body Scripts Settings

Body Cookies Headers 5 Test Results 200 OK 83 ms 765 B

{} JSON Preview Visualize


```
{
    "chain": [
        {
            "index": 1,
            "previous_hash": "1",
            "proof": 100,
            "timestamp": 1772641176.0711849,
            "transactions": []
        },
        {
            "index": 2,
            "previous_hash": "975115cbfd62c86f6047a831e992d94abbe1117f76f628d699e4d9c81c4f1e00",
            "proof": 51030,
            "timestamp": 1772641243.2319267,
            "transactions": [
                {
                    "amount": 1.0,
                    "fee": 0,
                    "recipient": "3a4e655ab2aa433688474f105c205fbc",
                    "sender": "0"
                }
            ]
        },
        {
            "index": 3,
            "previous_hash": "6fac387ba553269fb753d60a8410460e832545345a06877ff29a27aaede4564d",
            "proof": 2094238,
            "timestamp": 1772641258.7109337,
            "transactions": [
                {
                    "amount": 1.0,
                    "fee": 0,
                    "recipient": "3a4e655ab2aa433688474f105c205fbc",
                    "sender": "0"
                }
            ]
        }
    ],
    "length": 3
}
```


- **Tạo TX low fee:**

http://localhost:5555/transactions/new

POST http://localhost:5555/transactions/new Send

Docs Params Authorization Headers 9 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

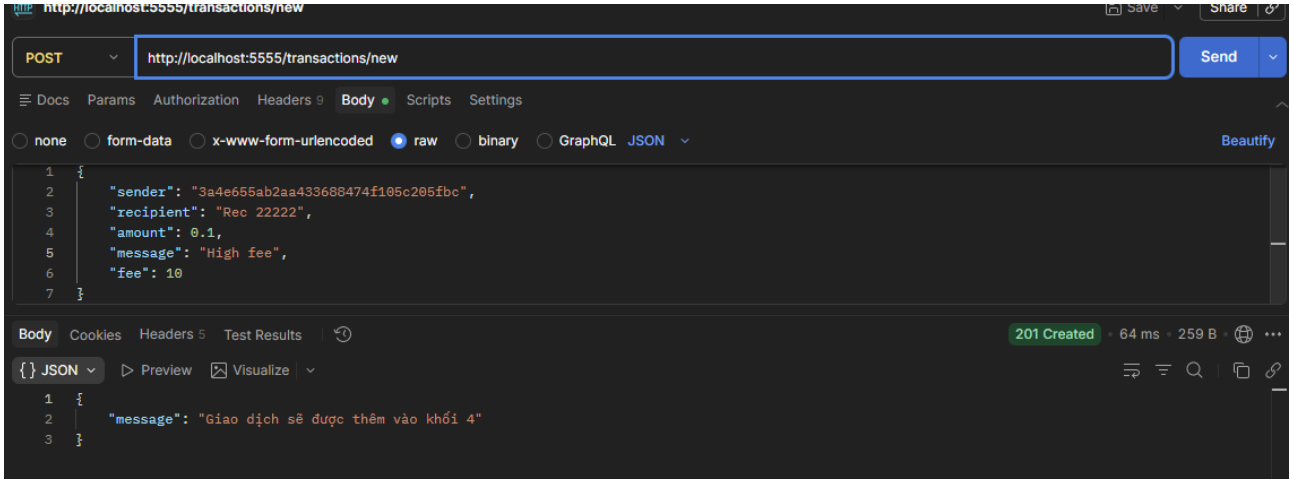

```
{
    "sender": "3a4e655ab2aa433688474f105c205fbc",
    "recipient": "Rec 1",
    "amount": 0.1,
    "message": "Low fee",
    "fee": 1
}
```


Body Cookies Headers 5 Test Results 201 Created 66 ms 259 B

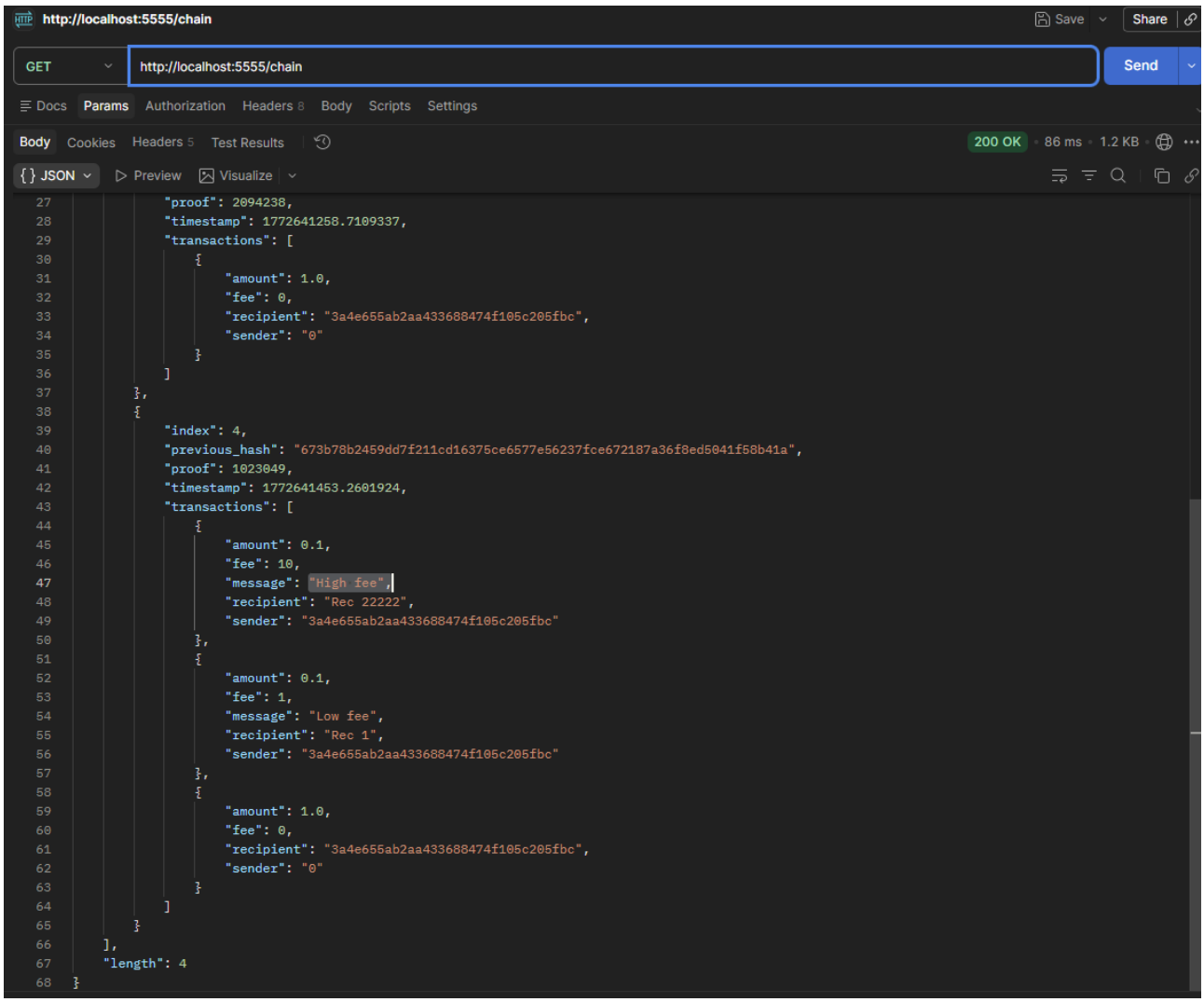
{} JSON Preview Visualize


```
{
    "message": "Giao dịch sẽ được thêm vào khối 4"
}
```


- **Tạo TX higher fee sau:**



- **Mine lên → thấy giao dịch có fee cao hơn sẽ được thực hiện trước dù tạo sau:**



3. Source code:

https://github.com/gianghitran/Blockchain_UIT/tree/main/Lab1

HẾT